

```

import os
import sys
from scapy.all import ARP, Ether, srp

def scan_network(ip_range):
    """
    Scans the local network for active devices using ARP requests.

    :param ip_range: The IP range to scan (e.g., "192.168.1.1/24")
    """
    print(f"Starting network scan on target range: {ip_range}")

    # Create an ARP request packet to ask "Who has this IP?"
    arp_request = ARP(pdst=ip_range)

    # Create an Ethernet broadcast frame to send the request to
    everyone on the local network
    broadcast_frame = Ether(dst="ff:ff:ff:ff:ff:ff")

    # Combine the Ethernet frame and ARP request into a single packet
    packet = broadcast_frame / arp_request

    try:
        # Send the packet and wait for responses (srp = Send and
        Receive Packets at Layer 2)
        # timeout: time to wait for a response in seconds
        # verbose: set to False to prevent Scapy from printing its
        inner status messages
        answered_list = srp(packet, timeout=2, verbose=False)[0]
    except PermissionError:
        print("\n[!] Error: Root/Administrator privileges are required
        to run a Scapy scan.")
        print("Please re-run this script using 'sudo' (Linux/macOS) or
        as Administrator (Windows).")
        return
    except Exception as e:
        print(f"\n[!] An error occurred during transmission: {e}")
        return

    # Parse and display the results
    print("\nActive devices found on the network:")
    print("-----")
    print(f"{'IP Address':<16} | {'MAC Address':<18}")
    print("-----")

    discovered_devices = []
    for element in answered_list:
        # Extract the source IP and source MAC address from the

```

```

response packet
    device_info = {
        "ip": element[1].psrc,
        "mac": element[1].hwsrc
    }
    discovered_devices.append(device_info)
    print(f"{device_info['ip']:<16} | {device_info['mac']:<18}")

    print("-----")
    print(f"Scan complete. Total active devices discovered:
{len(discovered_devices)}")

if __name__ == "__main__":
    # Standard home network default range example
    # Change this to your specific subnet mask if it differs (e.g.,
10.0.0.1/24)
    target_subnet = input("Enter target IP range (e.g.,
192.168.1.1/24): ").strip()

    if target_subnet:
        scan_network(target_subnet)
    else:
        print("Invalid input. Exiting.")

```